

CS-334: Machine Learning

Lecture 05: Gradient Descent

(with In-Class Exercise #4)

09/10/2026

Prof. Shengpu Tang



Announcements

- **HW1** due Sun 11:59pm on Gradescope
 - Autograder: you can submit multiple times

Important Note

- Please **assign pages** to relevant question parts
- You can do this before or **after the deadline**, it will not change your submission time
- If we cannot locate your answer because your pdf is not tagged with relevant pages, we reserve the right to not grade your answer or **deduct points**

Perceptron

a mistake-driven algorithm

$$k = 0, \vec{\theta}^{(0)} = \vec{0}, b^{(0)} = 0$$

while not all points are correctly classified

for $i = 1 \dots N$

$$\text{if } y^{(i)} (\vec{\theta}^{(k)} \cdot \vec{x}^{(i)} + b^{(k)}) \leq 0$$

$$\vec{\theta}^{(k+1)} = \vec{\theta}^{(k)} + y^{(i)} \vec{x}^{(i)}$$

$$b^{(k+1)} = b^{(k)} + y^{(i)}$$

$$k = k + 1$$

Deriving the update rule for offset
by augmenting the feature space

$$\vec{x}' = [1, \vec{x}]^T \quad \vec{\theta}' = [b, \vec{\theta}]^T$$

$$\vec{\theta}' \cdot \vec{x}' = b + \vec{\theta} \cdot \vec{x}$$

$$[b^{(k+1)}, \vec{\theta}^{(k+1)}]^T = [b^{(k)}, \vec{\theta}^{(k)}] + y^{(i)} [1, \vec{x}^{(i)}]^T$$

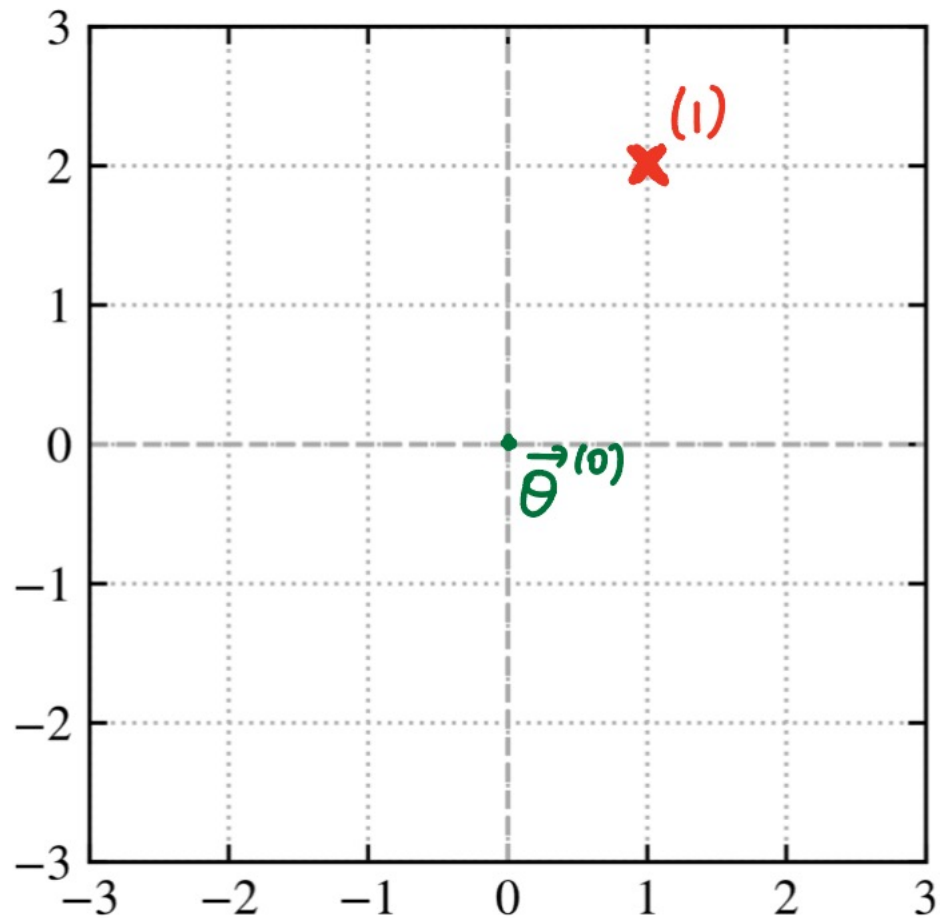
Perceptron: Properties

Convergence Theorem: If the training dataset is *linearly separable*, the perceptron is guaranteed to converge after a finite number of updates.

However:

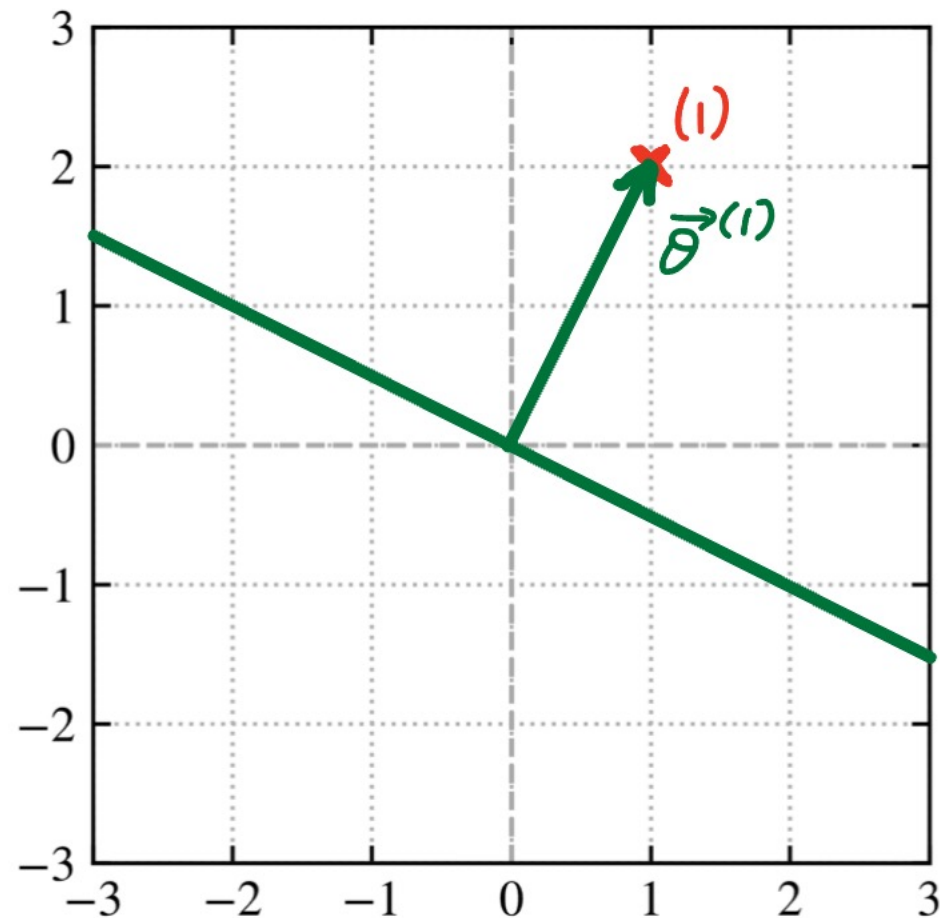
- No guarantee on uniqueness of solutions; depends on initialization and the order of data points
 - Some data points might not be used
- “finite number” of updates, but can be large
 - “undershoot” when current point remains misclassified after update
 - Previously correct data point may become misclassified after update

Initialization



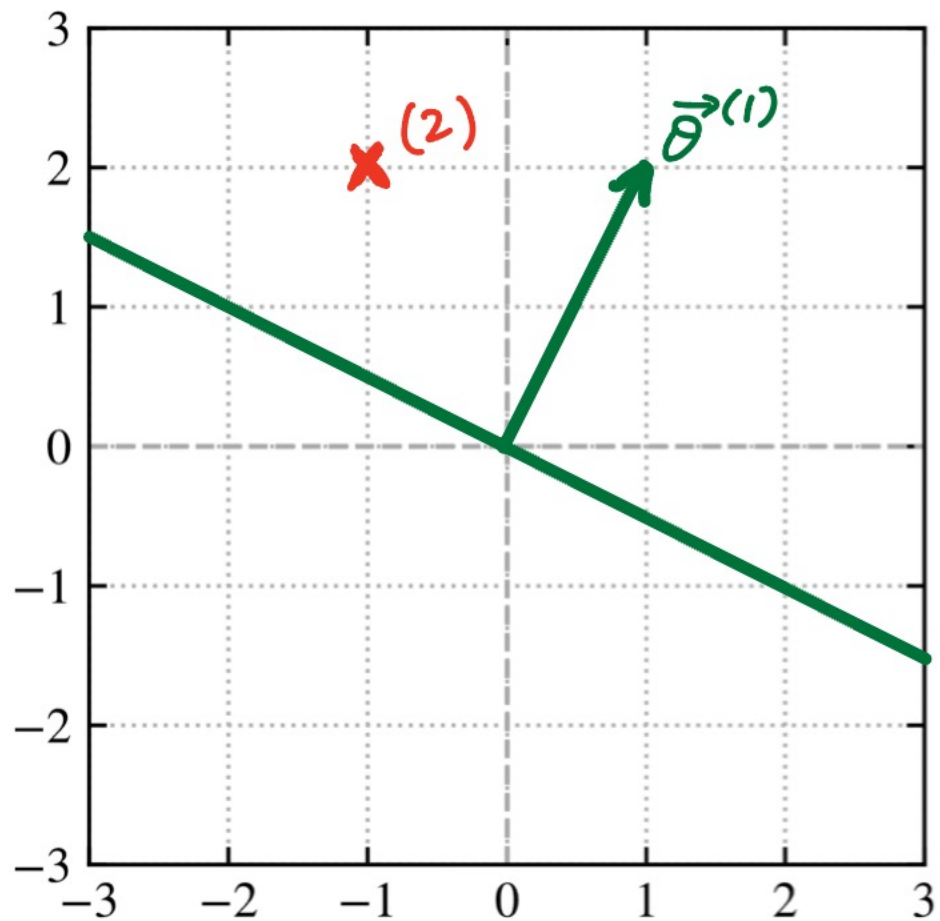
$$\vec{\theta}^{(0)} = [0, 0]$$

After first update

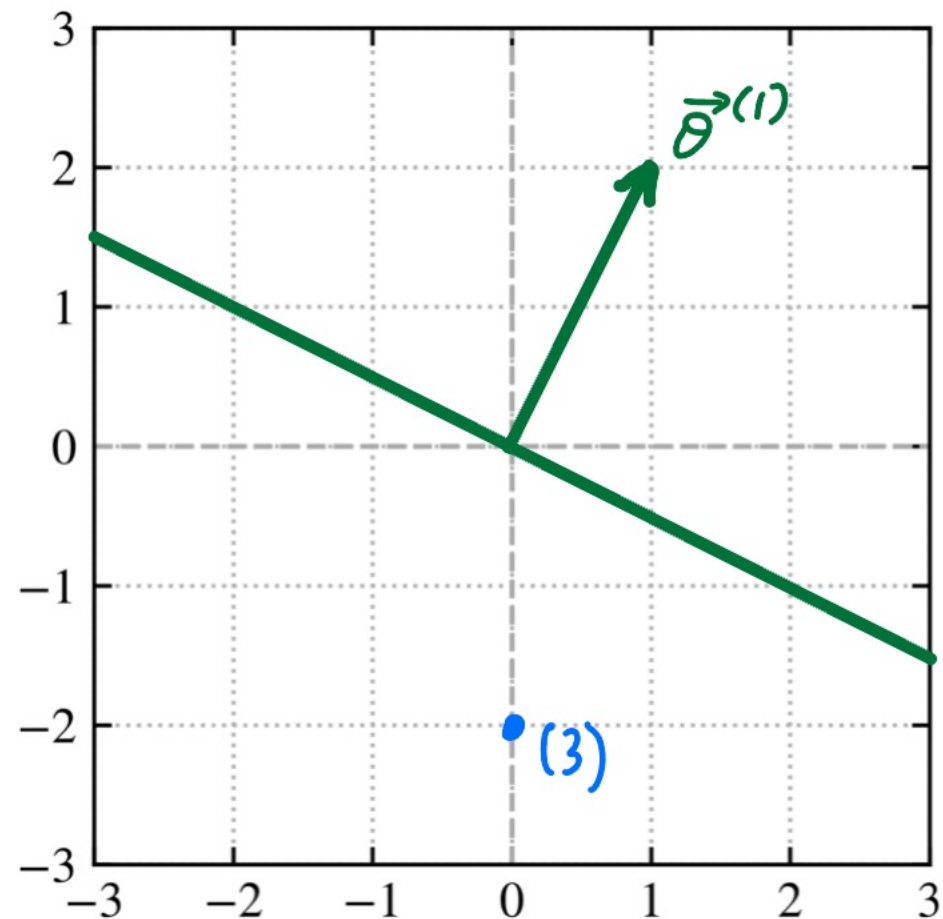


$$\vec{\theta}^{(1)} = [1, 2]$$

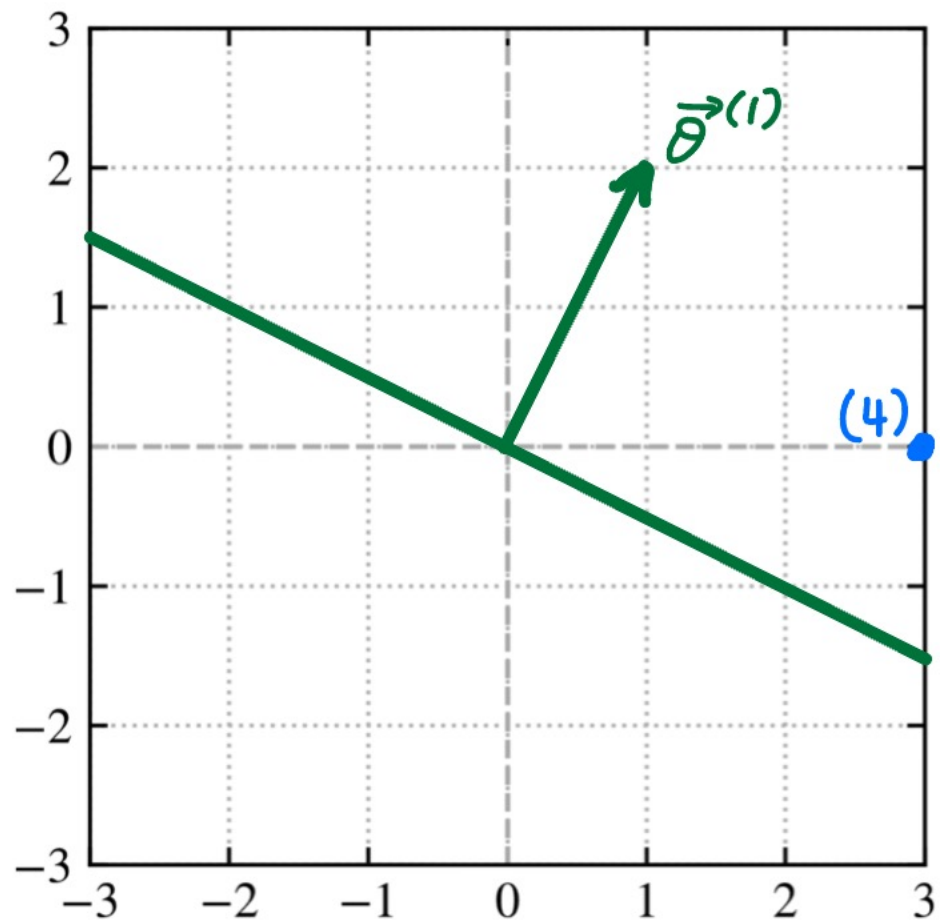
No mistake. No update.



No Mistake. No update.

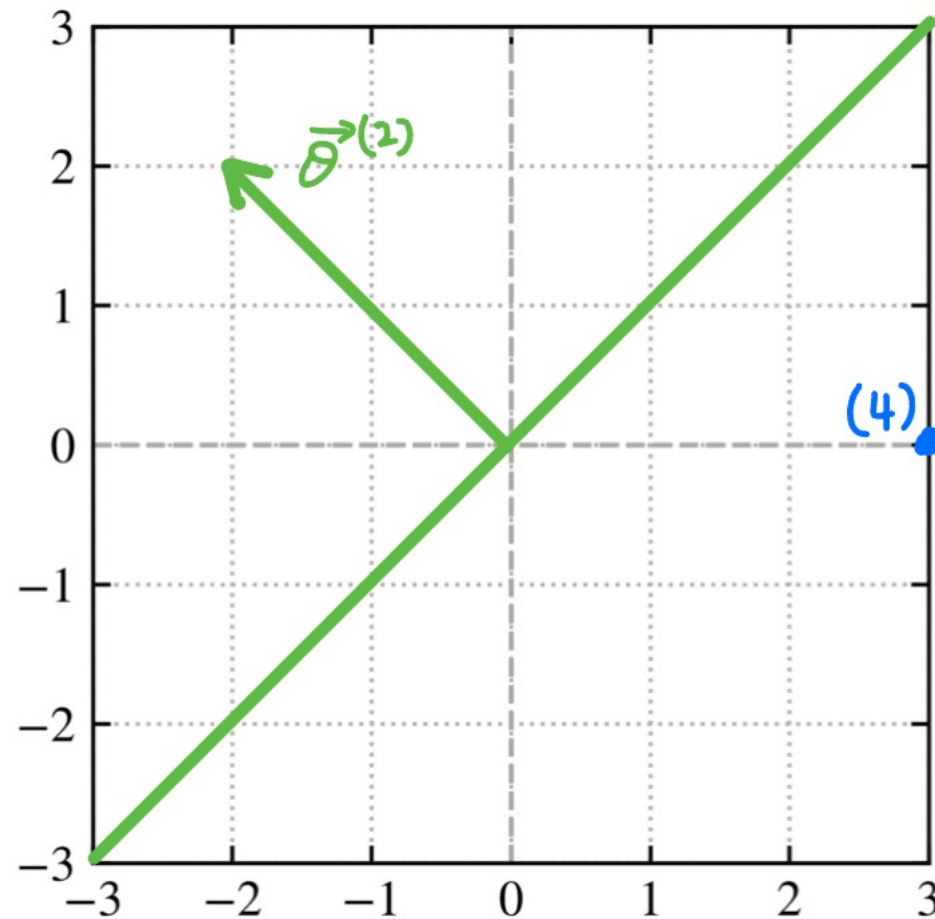


Mistake.



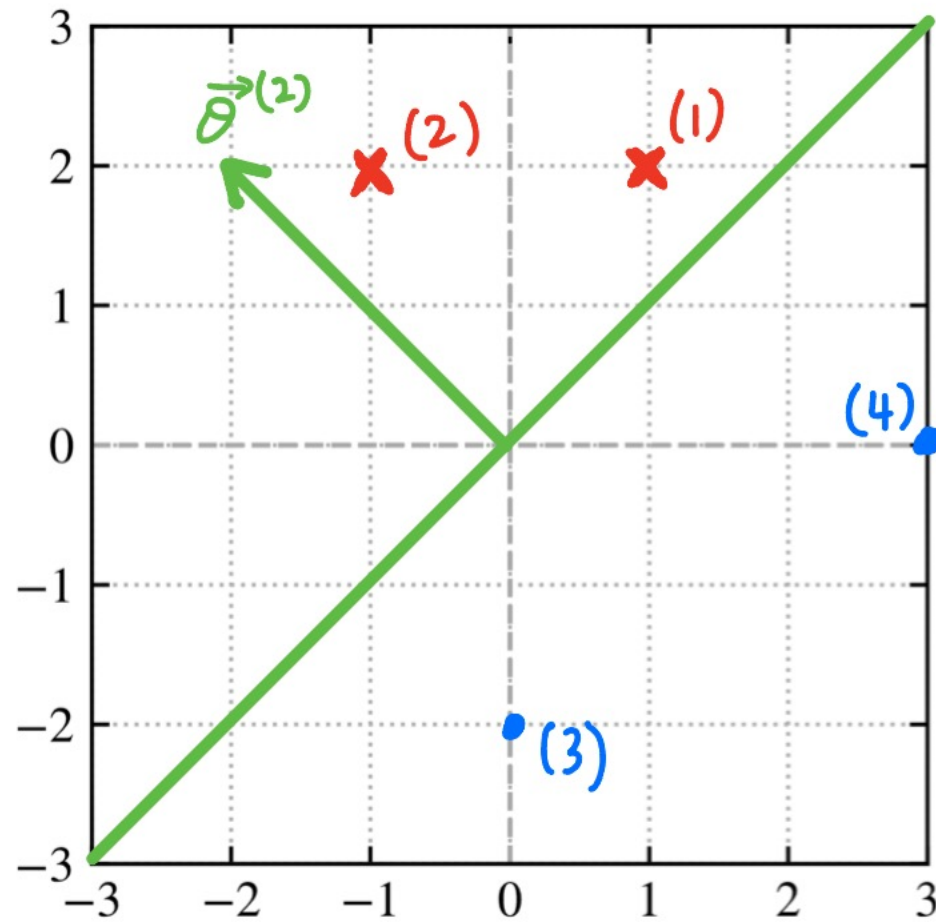
$$\vec{\theta}^{(1)} = [1, 2]$$

Update.

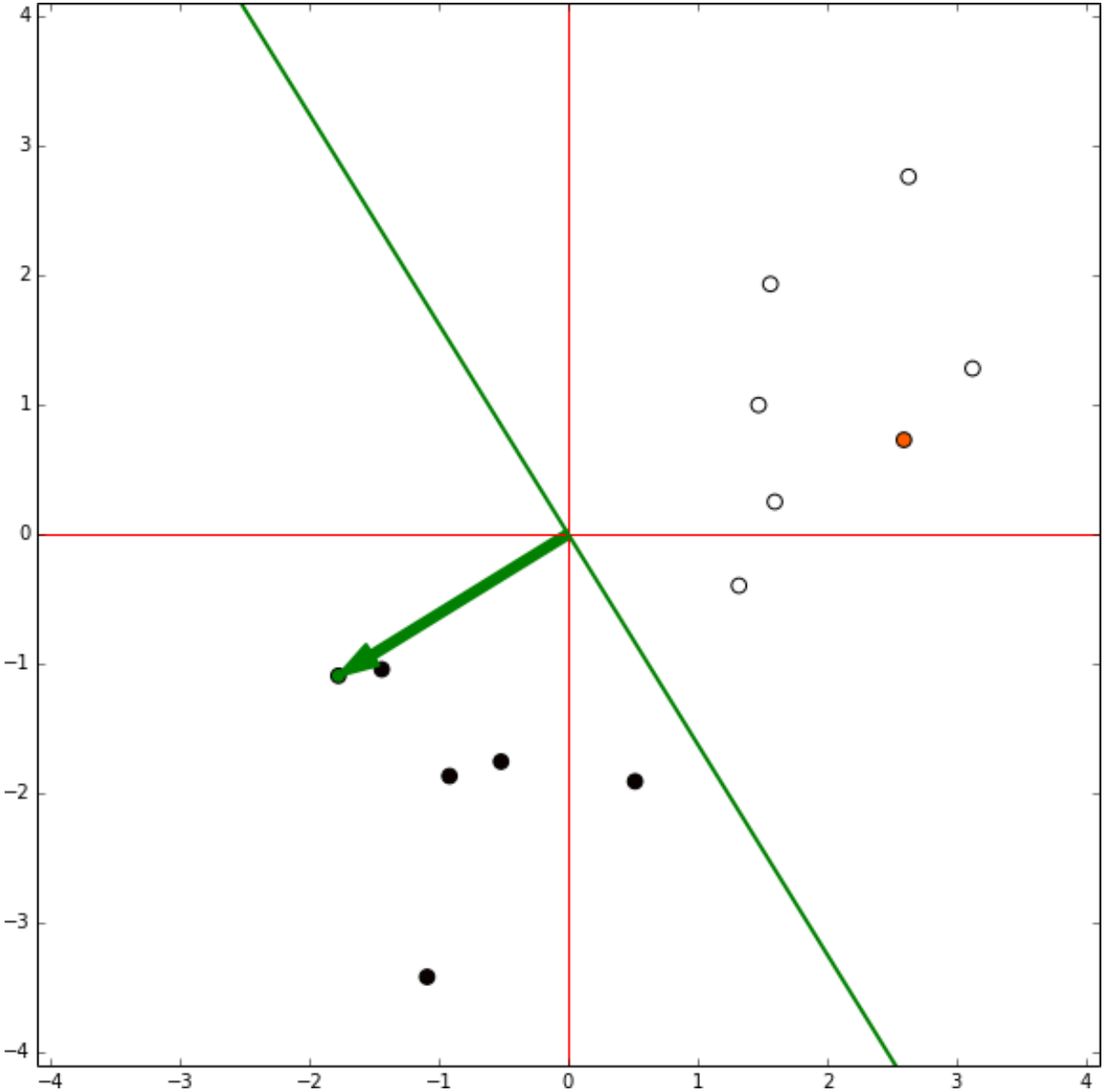


$$\vec{\theta}^{(2)} = [-2, 2]$$

Final Decision Boundary



$$\vec{\theta}^{(2)} = [-2, 2]$$

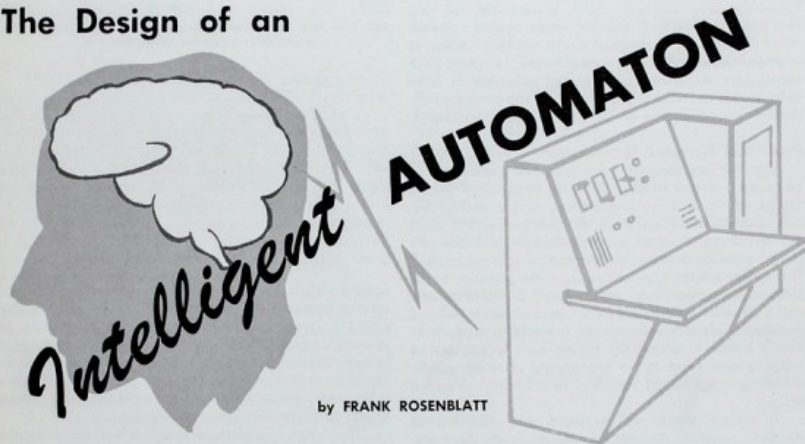


research trends

CORNELL AERONAUTICAL LABORATORY, INC., BUFFALO 21, NEW YORK



The Design of an



by FRANK ROSENBLATT

Introducing the perceptron — A machine which senses, recognizes, remembers, and responds like the human mind.

STORIES about the creation of machines having human qualities have long been a fascinating province in the realm of science fiction. Yet we are now about to witness the birth of such a machine — a machine capable of perceiving, recognizing, and identifying its surroundings without any human training or control.

Development of that machine has stemmed from a search for an understanding of the physical mechanisms which underlie human experience and intelligence. The question of the nature of these processes is at least as ancient as any other question in western science and philosophy, and, indeed, ranks as one of the greatest scientific challenges of our time.

Our understanding of this problem has gone perhaps as far as had the development of physics before Newton. We have some excellent descriptions of the phenomena to be explained, a number of interesting hypotheses, and a little detailed knowledge about events in the nervous system. But we lack agreement on any integrated set of principles by which the functioning of the nervous system can be understood.

We believe now that this ancient problem is about to yield to our theoretical investigation for three reasons:

First, in recent years our knowledge of the functioning of individual cells in the central nervous system has vastly increased.

Second, large numbers of engineers and mathematicians are, for the first time, undertaking serious study of the mathematical basis for thinking, perception, and the handling of information by the central nervous system, thus providing the hope that these problems may be within our intellectual grasp.

Third, recent developments in probability theory and in the mathematics of random processes provide new tools for the study of events in the nervous system, where only the gross statistical organization is known and the precise cell-by-cell "wiring diagram" may never be obtained.

Receives Navy Support

In July, 1957, Project PARA (Perceiving and Recognizing Automaton), an internal research program which had been in progress for over a year at Cornell Aeronautical Laboratory, received the support of the Office of Naval Research. The program had been concerned primarily with the application of probability theory to

***"Introducing the perceptron —
A machine which senses,
recognizes, remembers, and
responds like the human mind."
Research Trends, 1958***

THE BROOKLYN SAVINGS BANK

3 1/4%
ANNUAL DIVIDEND

SAVINGS BANK
BANK OF AMERICA
BANK OF NEW YORK
BANK OF THE CITY OF NEW YORK
BANK OF THE STATE OF NEW YORK
BANK OF THE STATE OF NEW JERSEY
BANK OF THE STATE OF PENNSYLVANIA
BANK OF THE STATE OF OHIO
BANK OF THE STATE OF MICHIGAN
BANK OF THE STATE OF ILLINOIS
BANK OF THE STATE OF INDIANA
BANK OF THE STATE OF WISCONSIN
BANK OF THE STATE OF MINNESOTA
BANK OF THE STATE OF IOWA
BANK OF THE STATE OF NEBRASKA
BANK OF THE STATE OF KANSAS
BANK OF THE STATE OF OKLAHOMA
BANK OF THE STATE OF ARIZONA
BANK OF THE STATE OF CALIFORNIA
BANK OF THE STATE OF TEXAS
BANK OF THE STATE OF LOUISIANA
BANK OF THE STATE OF MISSISSIPPI
BANK OF THE STATE OF ALABAMA
BANK OF THE STATE OF GEORGIA
BANK OF THE STATE OF FLORIDA
BANK OF THE STATE OF SOUTH CAROLINA
BANK OF THE STATE OF NORTH CAROLINA
BANK OF THE STATE OF VIRGINIA
BANK OF THE STATE OF MARYLAND
BANK OF THE STATE OF DELAWARE
BANK OF THE STATE OF PENNSYLVANIA
BANK OF THE STATE OF OHIO
BANK OF THE STATE OF MICHIGAN
BANK OF THE STATE OF ILLINOIS
BANK OF THE STATE OF INDIANA
BANK OF THE STATE OF WISCONSIN
BANK OF THE STATE OF MINNESOTA
BANK OF THE STATE OF IOWA
BANK OF THE STATE OF NEBRASKA
BANK OF THE STATE OF KANSAS
BANK OF THE STATE OF OKLAHOMA
BANK OF THE STATE OF ARIZONA
BANK OF THE STATE OF CALIFORNIA
BANK OF THE STATE OF TEXAS
BANK OF THE STATE OF LOUISIANA
BANK OF THE STATE OF MISSISSIPPI
BANK OF THE STATE OF ALABAMA
BANK OF THE STATE OF GEORGIA
BANK OF THE STATE OF FLORIDA
BANK OF THE STATE OF SOUTH CAROLINA
BANK OF THE STATE OF NORTH CAROLINA
BANK OF THE STATE OF VIRGINIA
BANK OF THE STATE OF MARYLAND
BANK OF THE STATE OF DELAWARE

NEW NAVY DEVICE LEARNS BY DOING

Psychologist Shows Embryo of Computer Designed to Read and Grow Wiser

By ALBERT CAMUS

WASHINGTON, July 7 (UPI)—The Navy revealed the embryo of an electronic computer today that it expects will be able to walk, talk, see, write, reproduce itself and be conscious of its existence.

The embryo—the Weather Bureau's \$2,000,000 "704" computer—learned to differentiate between right and left after fifty attempts in the Navy's demonstration for newsmen.

The service said it would use this principle to build the first of its Perceptron thinking machines that will be able to read and write. It is expected to be finished in about a year at a cost of \$100,000.

Dr. Frank Rosenblatt, designer of the Perceptron, conducted the demonstration. He said the machine would be the first device to think as the human brain. As do human beings, Perceptron will make mistakes at first, but will grow wiser as it gains experience, he said.

Dr. Rosenblatt, a research psychologist at the Cornell Aeronautical Laboratory, Buffalo, said Perceptrons might be fired to the planets as mechanical space explorers.

Books of The Times

By ALBERT CAMUS

WASHINGTON, July 7 (UPI)—The Navy revealed the embryo of an electronic computer today that it expects will be able to walk, talk, see, write, reproduce itself and be conscious of its existence.

The embryo—the Weather Bureau's \$2,000,000 "704" computer—learned to differentiate between right and left after fifty attempts in the Navy's demonstration for newsmen.

The service said it would use this principle to build the first of its Perceptron thinking machines that will be able to read and write. It is expected to be finished in about a year at a cost of \$100,000.

Dr. Frank Rosenblatt, designer of the Perceptron, conducted the demonstration. He said the machine would be the first device to think as the human brain. As do human beings, Perceptron will make mistakes at first, but will grow wiser as it gains experience, he said.

Dr. Rosenblatt, a research psychologist at the Cornell Aeronautical Laboratory, Buffalo, said Perceptrons might be fired to the planets as mechanical space explorers.

Just Published!

ALGERIA
The Realities
by Germaine Tillion

ALBERT CAMUS

AND FOUR TO GO

NATIONAL BEST SELLER

MURDER

by REX STOUT

Elizabeth CADELL

by REX STOUT

SHADOWS ON THE WATER

by REX STOUT

ONCE TO SINAI

by REX STOUT

Ask Any Girl

by REX STOUT

Week after Week

By ALBERT CAMUS

NEW NAVY DEVICE LEARNS BY DOING

Psychologist Shows Embryo of Computer Designed to Read and Grow Wiser

WASHINGTON, July 7 (UPI)—The Navy revealed the embryo of an electronic computer today that it expects will be able to walk, talk, see, write, reproduce itself and be conscious of its existence.

The embryo—the Weather Bureau's \$2,000,000 "704" computer—learned to differentiate between right and left after fifty attempts in the Navy's demonstration for newsmen.

The service said it would use this principle to build the first of its Perceptron thinking machines that will be able to read and write. It is expected to be finished in about a year at a cost of \$100,000.

Dr. Frank Rosenblatt, designer of the Perceptron, conducted the demonstration. He said the machine would be the first device to think as the human brain. As do human beings, Perceptron will make mistakes at first, but will grow wiser as it gains experience, he said.

Dr. Rosenblatt, a research psychologist at the Cornell Aeronautical Laboratory, Buffalo, said Perceptrons might be fired to the planets as mechanical space explorers.

Without Human Controls

The Navy said the perceptron would be the first non-living mechanism "capable of receiving, recognizing and identifying its surroundings without any human training or control."

The "brain" is designed to remember images and information it has perceived itself. Ordinary computers remember only what is fed into them on punch cards or magnetic tape.

Later Perceptrons will be able to recognize people and call out their names and instantly translate speech in one language to speech or writing in another language, it was predicted.

Mr. Rosenblatt said in principle it would be possible to build brains that could reproduce themselves on an assembly line and which would be conscious of their existence.

In today's demonstration, the "704" was fed two cards, one with squares marked on the left side and the other with squares on the right side.

Learns by Doing

In the first fifty trials, the machine made no distinction between them. It then started registering a "Q" for the left squares and "O" for the right squares.

Dr. Rosenblatt said he could explain why the machine learned only in highly technical terms. But he said the computer had undergone a "self-induced change in the wiring diagram."

The first Perceptron will have about 1,000 electronic "association cells" receiving electrical impulses from an eye-like scanning device with 400 photo-cells. The human brain has 10,000,000,000 responsive cells, including 100,000,000 connections with the eyes.

"...the embryo of an electronic computer that [was expected] to walk, talk, see, write, reproduce itself, and be conscious of its existence..."

The New York Times, 1958

Mark 1 Perceptron

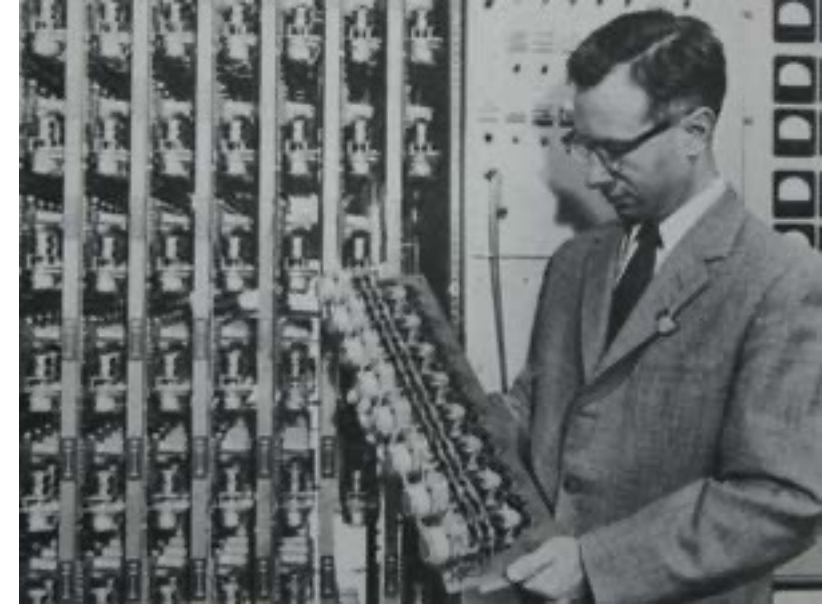
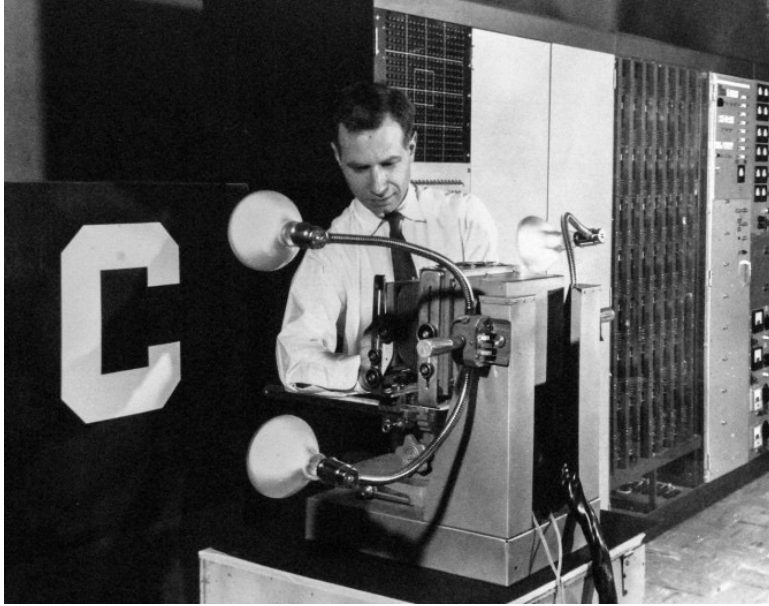
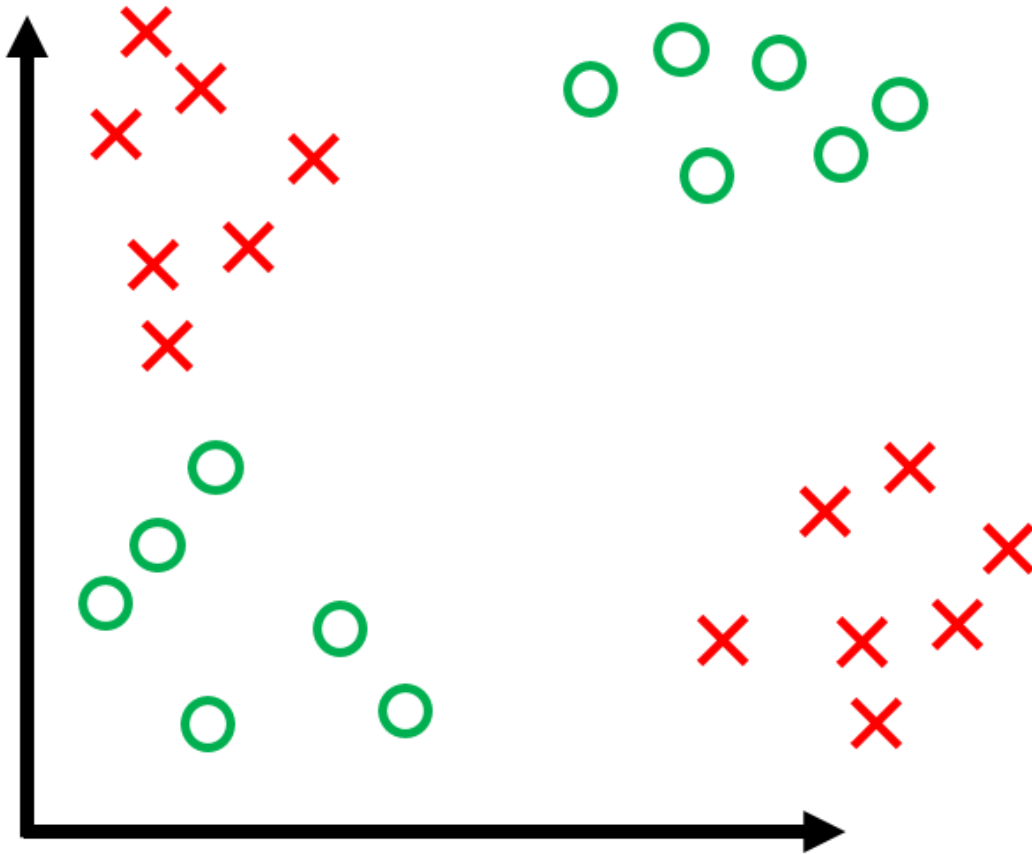


Figure 4.8 Illustration of the Mark 1 perceptron hardware. The photograph on the left shows how the inputs were obtained using a simple camera system in which an input scene, in this case a printed character, was illuminated by powerful lights, and an image focussed onto a 20×20 array of cadmium sulphide photocells, giving a primitive 400 pixel image. The perceptron also had a patch board, shown in the middle photograph, which allowed different configurations of input features to be tried. Often these were wired up at random to demonstrate the ability of the perceptron to learn without the need for precise wiring, in contrast to a modern digital computer. The photograph on the right shows one of the racks of adaptive weights. Each weight was implemented using a rotary variable resistor, also called a potentiometer, driven by an electric motor thereby allowing the value of the weight to be adjusted automatically by the learning algorithm.



https://youtu.be/cNxadbrN_al

What happens if not linearly separable?



The “famous XOR problem”

Minsky and Papert, “*Perceptrons*”, 1969.

The perceptron algorithm would run indefinitely and never converge, and is not guaranteed to find a solution that minimizes training error

Optimization Objective

Training Error

$$\mathcal{E}_N(\vec{\theta}) = \frac{1}{N} \sum_{i=1}^N \mathbb{1}[y^{(i)} (\vec{\theta} \cdot \vec{x}^{(i)}) \leq 0]$$

indicator function



for conciseness, we assume the offset parameter is implicit

What does $z = y(\vec{\theta} \cdot \vec{x})$ tell us?

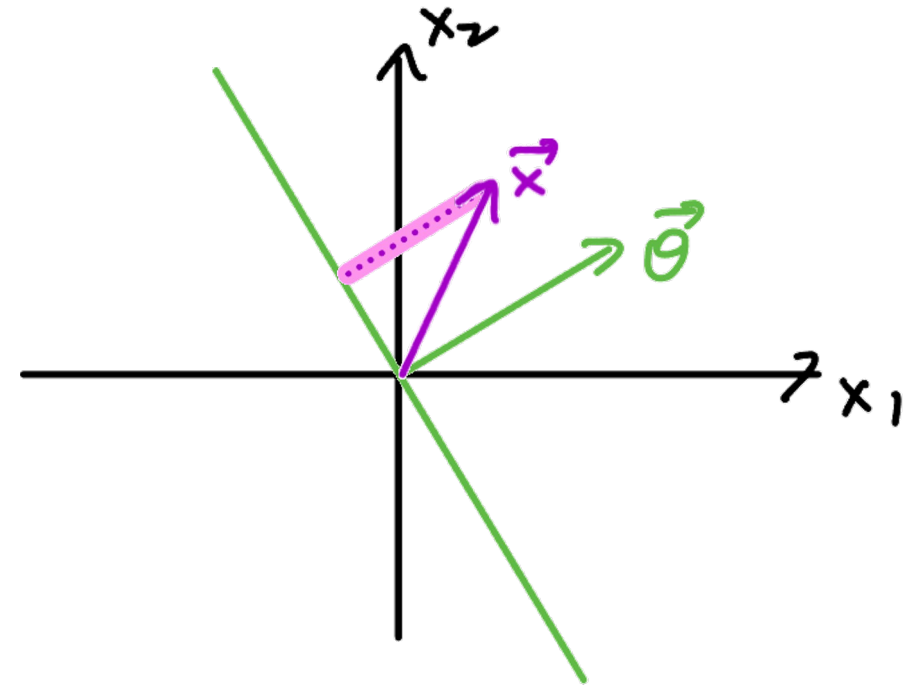
Sign: whether right or wrong
 >0 <0

Magnitude: how right or wrong

$$y = \pm 1 \text{ so } |y| = 1$$

$|y(\vec{\theta} \cdot \vec{x})| = |\vec{\theta} \cdot \vec{x}|$ is proportional to

the distance from decision boundary $\frac{|\vec{\theta} \cdot \vec{x}|}{\|\vec{\theta}\|}$



Changing Optimization Objective

Training Error

$$\mathcal{E}_N(\vec{\theta}) = \frac{1}{N} \sum_{i=1}^N \mathbb{1}[y^{(i)} (\vec{\theta} \cdot \vec{x}^{(i)}) \leq 0]$$

indicator function



for conciseness, we assume the offset parameter is implicit

Changing Optimization Objective

Training Error

$$\mathcal{E}_N(\vec{\theta}) = \frac{1}{N} \sum_{i=1}^N \mathbb{1}[y^{(i)} (\vec{\theta} \cdot \vec{x}^{(i)}) \leq 0]$$

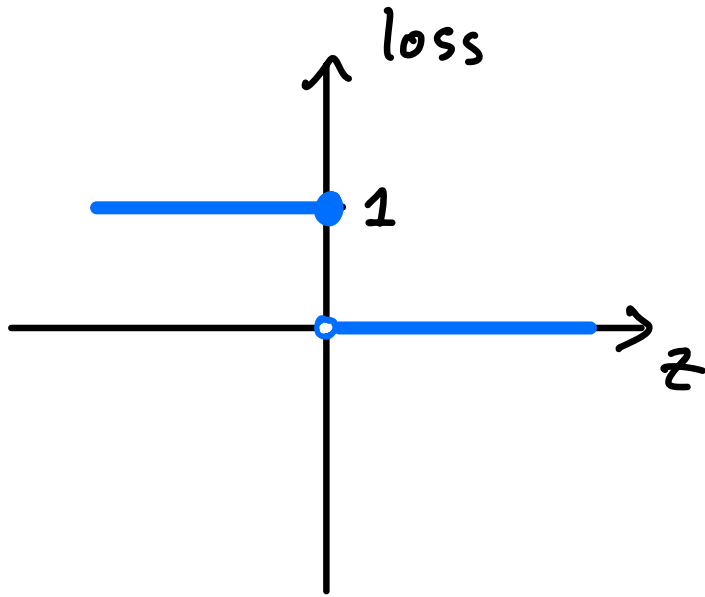
Empirical Risk: a “relaxed” version of training error

$$R_N(\vec{\theta}) = \frac{1}{N} \sum_{i=1}^N \text{loss}(y^{(i)}, \vec{\theta} \cdot \vec{x}^{(i)})$$

Empirical Risk

$$R_N(\vec{\theta}) = \frac{1}{N} \sum_{i=1}^N \text{loss}(y^{(i)}, \vec{\theta} \cdot \vec{x}^{(i)})$$

Zero-one loss: $\text{loss}_{0-1}(z) = \mathbb{1}[z \leq 0]$ $z = y(\vec{\theta} \cdot \vec{x})$
(empirical risk with zero-one loss --> training error)



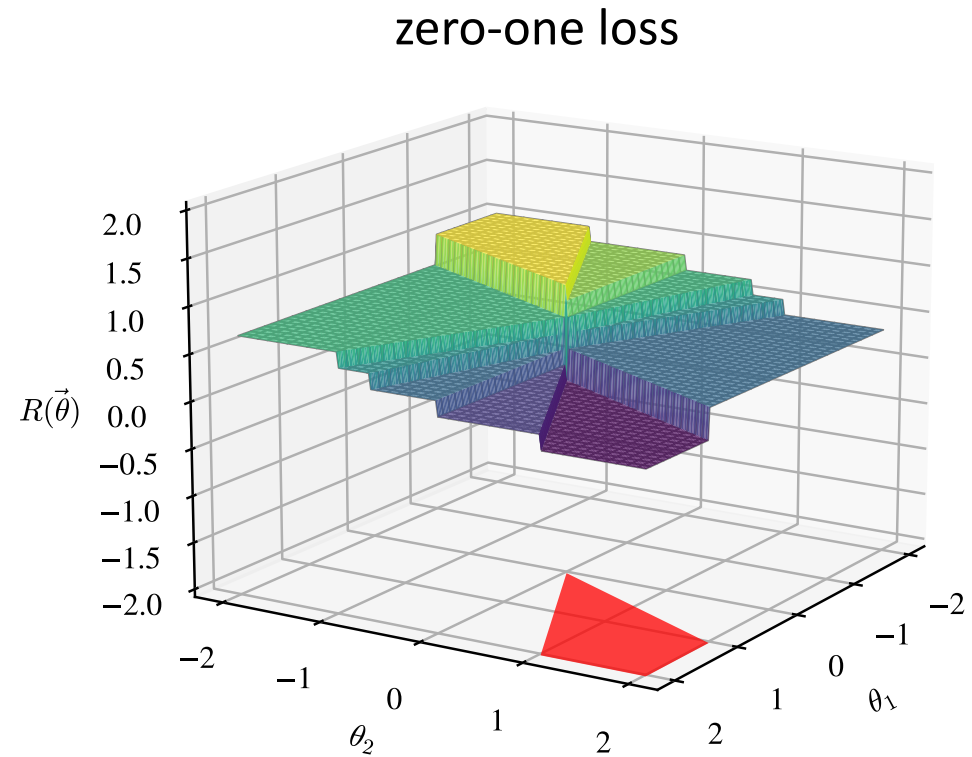
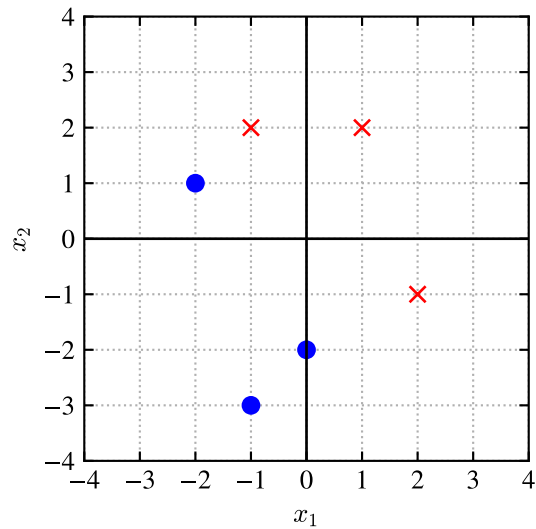
Issues:

- not continuous
- not differentiable
- not convex

direct minimization of empirical risk with 0-1 loss
is challenging for the general case (NP hard)

Loss Surface

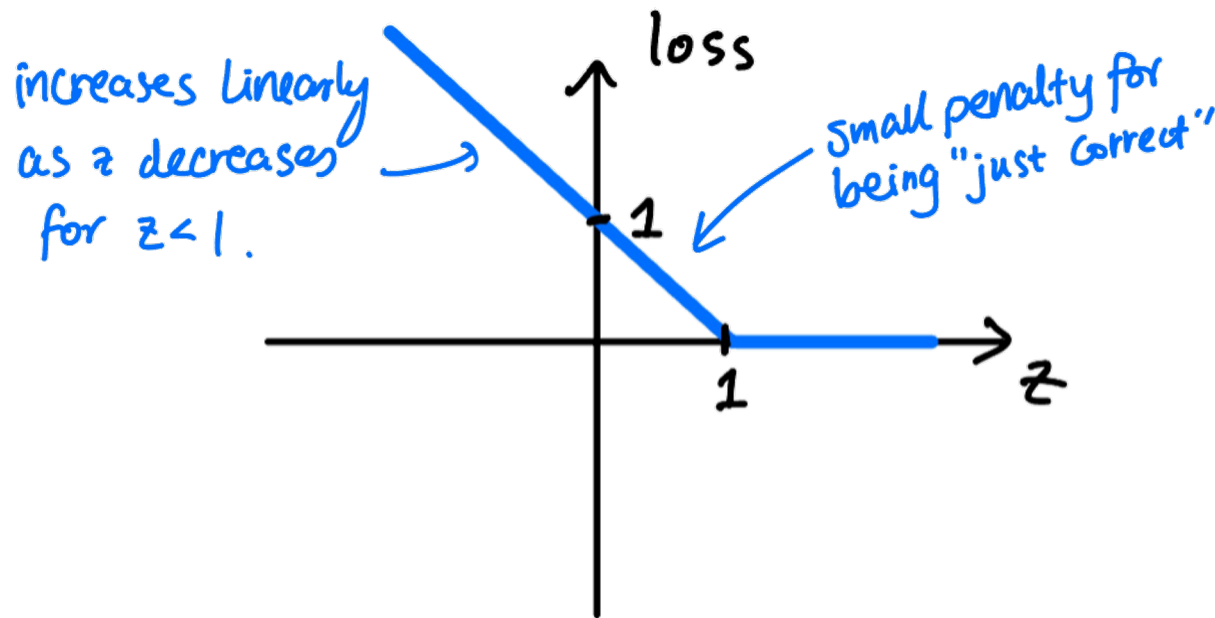
$$R_N(\vec{\theta}) = \frac{1}{N} \sum_{i=1}^N \text{loss}(y^{(i)}, \vec{\theta} \cdot \vec{x}^{(i)})$$



Empirical Risk

$$R_N(\vec{\theta}) = \frac{1}{N} \sum_{i=1}^N \text{loss}(y^{(i)}, \vec{\theta} \cdot \vec{x}^{(i)})$$

Hinge loss: $\text{loss}_h(z) = \max\{0, 1 - z\}$ $z = y(\vec{\theta} \cdot \vec{x})$



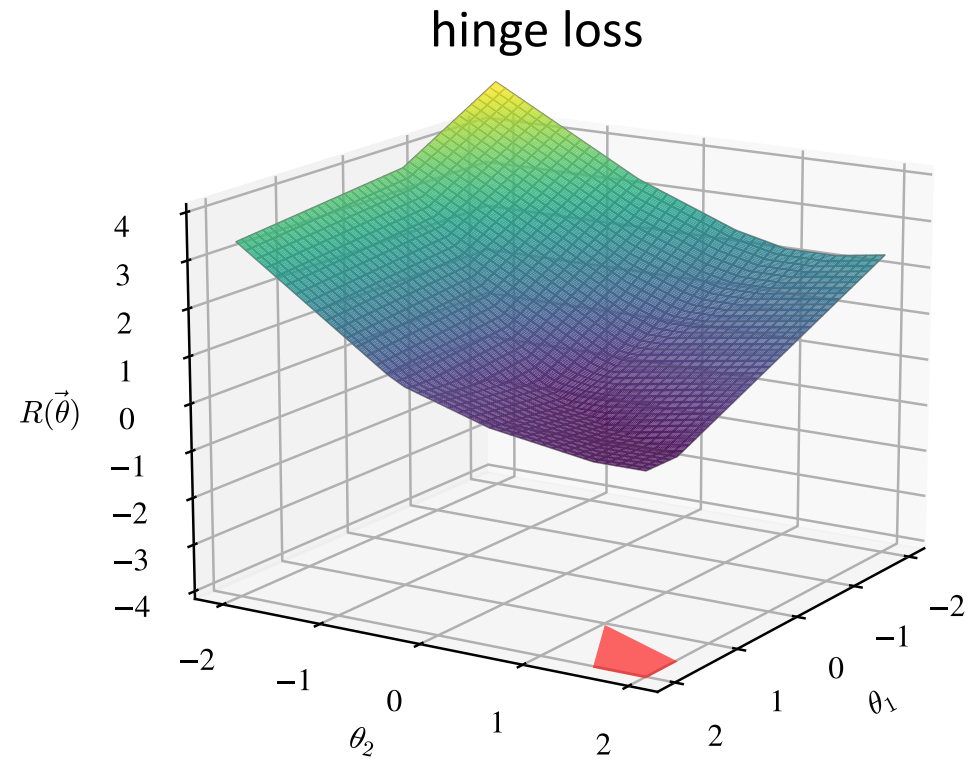
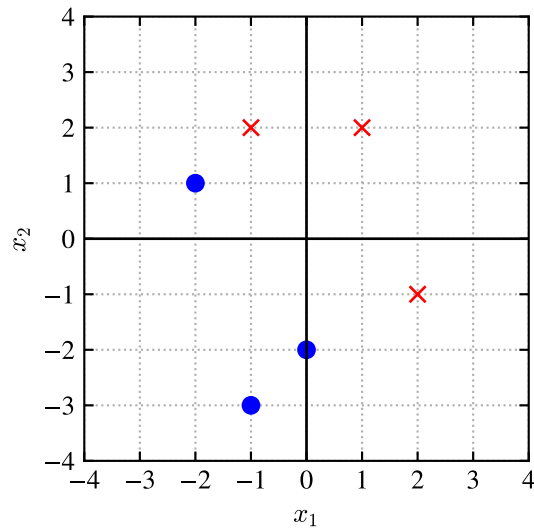
(Nicer) Properties:

- continuous
- differentiable
- convex

There exists simple algorithms that can optimize this

Loss Surface

$$R_N(\vec{\theta}) = \frac{1}{N} \sum_{i=1}^N \text{loss}(y^{(i)}, \vec{\theta} \cdot \vec{x}^{(i)})$$



Link's Foggy Descent



Link's Foggy Descent



Link's Foggy Descent

- You're standing at the summit of **Mount Lanayru**.
- Suddenly, a **thick fog** rolls in. You can't see the shrines, the towers, or even the path right in front. You don't have a compass, and your *Sheikah Slate* is broken.
- You can only feel the ground around your feet.
- How would you get to the campfire at the very bottom of the valley?



Link's Foggy Descent

- Look for the direction of steepest descent
- Take a small step
- Repeat



Gradient

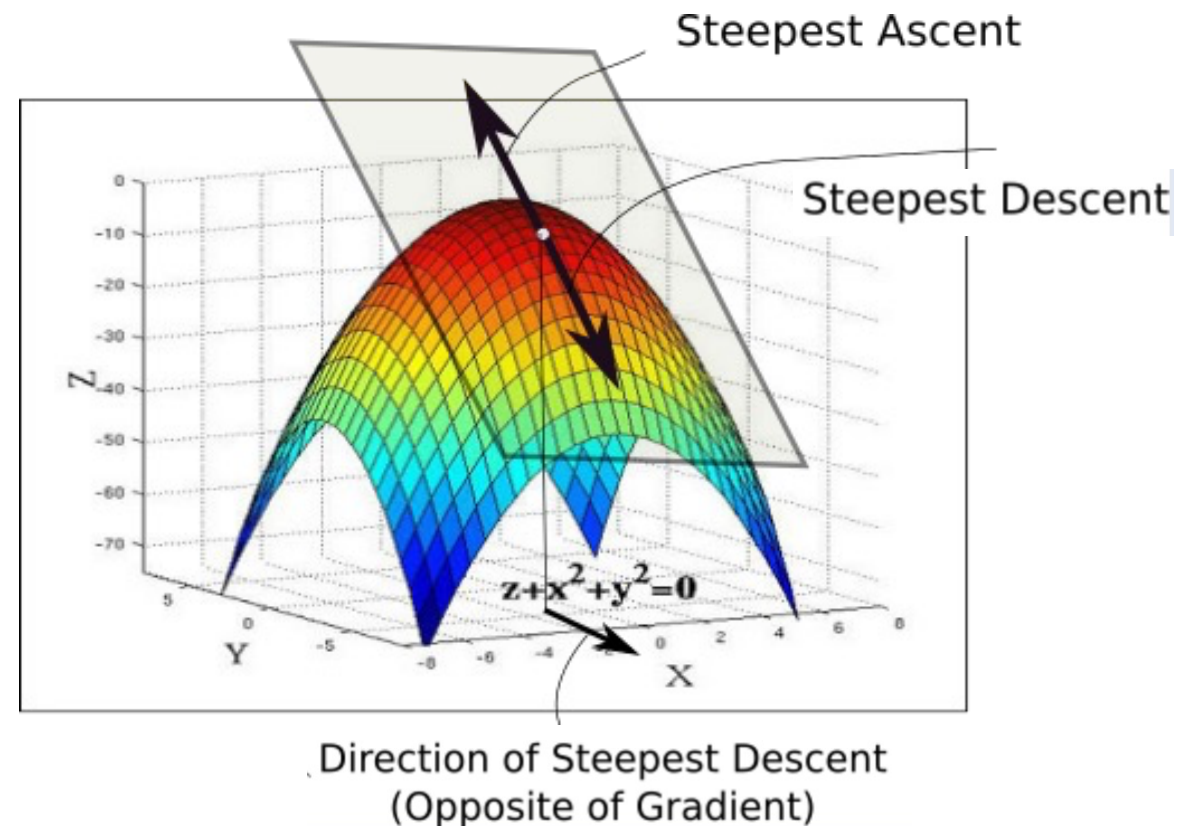
$f(\vec{\theta}) : \mathbb{R}^d \rightarrow \mathbb{R}$ is some function

We want to find a $\vec{\theta}^*$ that minimizes $f(\vec{\theta})$

Gradient captures the “local shape” of the function

$$\nabla_{\vec{\theta}} f(\vec{\theta}) : \mathbb{R}^d \rightarrow \mathbb{R}^d$$

for every $\vec{\theta}$, it tells us which way to go such that $f(\vec{\theta})$ increases the fastest

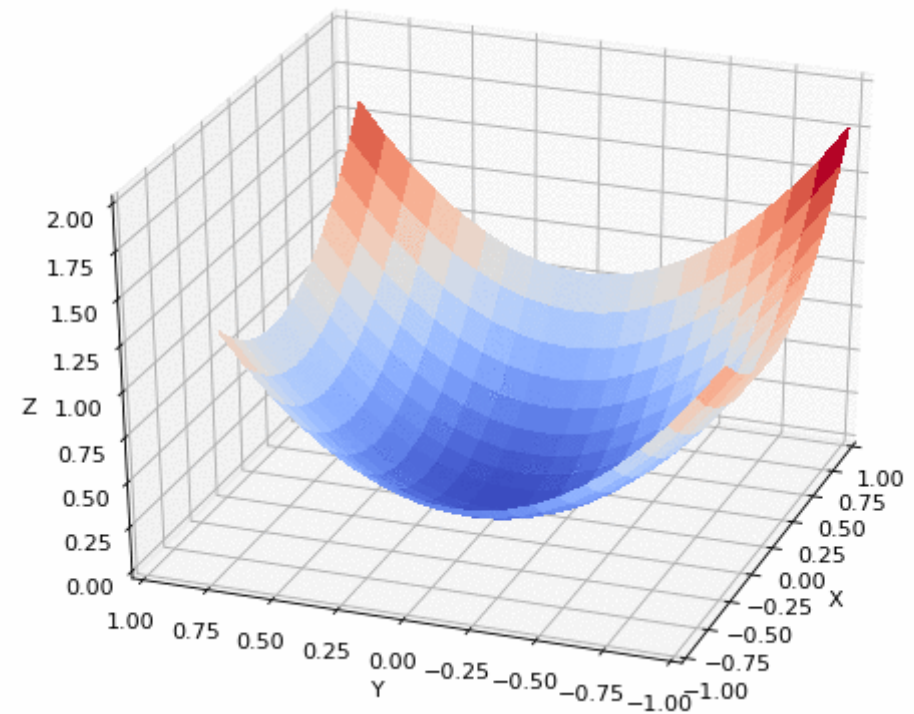


Gradient Descent

$f(\vec{\theta}) : \mathbb{R}^d \rightarrow \mathbb{R}$ is some function

We want to find a $\vec{\theta}^*$ that minimizes $f(\vec{\theta})$

- Start somewhere
- Take a small step in the opposite direction of the gradient
- Repeat



Gradient Descent

Goal: minimize an arbitrary function $f(\vec{\theta})$, $f: \mathbb{R}^d \rightarrow \mathbb{R}$

$$k = 0, \vec{\theta}^{(0)} = \vec{0}$$

while not converged

$$\vec{\theta}^{(k+1)} = \vec{\theta}^{(k)} - \eta_k \nabla_{\vec{\theta}} f(\vec{\theta})|_{\vec{\theta}=\vec{\theta}^{(k)}}$$

$$k = k + 1$$

← stopping criteria

← step size
(learning rate)

Lecture 5. Gradient Descent

Suppose we want to minimize some $f(\vec{\theta})$ with respect to $\vec{\theta}$.
 We know gradient $\nabla_{\vec{\theta}} f(\vec{\theta})$ points in the direction of steepest increase.
 Idea: take a small step in the opposite direction.

convergence criteria

$$\vec{\theta}^{(0)} = \vec{\theta}, k = 0$$

While not converged

$$\vec{\theta}^{(k+1)} = \vec{\theta}^{(k)} - \eta_k \nabla_{\vec{\theta}} f(\vec{\theta}) \Big|_{\vec{\theta} = \vec{\theta}^{(k)}}$$

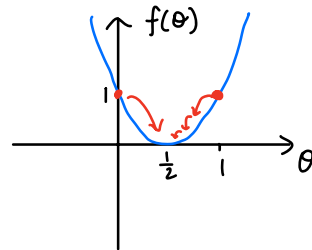
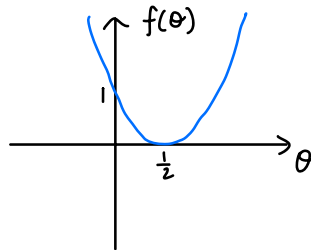
learning rate

"evaluated at" the current $\vec{\theta}$

$k++$

Warm-up: one-dimensional GD.

Let $f(\theta) = \theta^2 - \theta + 1$. Find minimizer θ^* of $f(\theta)$, $\theta^* = \operatorname{argmin} f(\theta)$.



We'll come back to this.
 it's not always possible
 to solve analytically.

► How would you solve it in Calc 1? $f'(\theta) = 2\theta - 1 = 0$, $\theta^* = \frac{1}{2}$.

► How to solve using GD?
 What is the update rule?

$\theta^{(0)} = 0$, $k = 0$
 While not converged

$$\theta^{(k+1)} = \theta^{(k)} - \eta_k \underbrace{f'(\theta)}_{(2\theta - 1)} \Big|_{\theta = \theta^{(k)}}$$

$k++$

• if $\theta^{(0)} = 0$, $\eta = \frac{1}{2}$,
 $\theta^{(0)} = 0$, $\theta^{(1)} = \frac{1}{2}$, $\theta^{(2)} = \frac{1}{2}$, converged.

• if $\theta^{(0)} = 1$, $\eta = \frac{1}{2}$,
 $\theta^{(0)} = 1$, $\theta^{(1)} = \frac{1}{2}$, $\theta^{(2)} = \frac{1}{2}$, converged.

• if $\theta^{(0)} = 1$, $\eta = \frac{1}{4}$
 $1 \rightarrow \frac{3}{4} \rightarrow \frac{5}{8} \rightarrow \frac{9}{16} \rightarrow \frac{17}{32} \rightarrow \dots \rightarrow \frac{2^k + 1}{2^{k+1}}$

$$\lim_{k \rightarrow \infty} \frac{2^k + 1}{2^{k+1}} = \lim_{k \rightarrow \infty} \left(\frac{1}{2} + \frac{1}{2^{k+1}} \right) = \frac{1}{2}$$

⊛ What's the effect of learning rate?
 How do we decide convergence?

Multiple dimensions

- Univariate function $f(\theta)$, $\theta \in \mathbb{R}$, $f: \mathbb{R} \rightarrow \mathbb{R}$

We need to find the derivative, $f'(\theta) = \frac{d}{d\theta} f(\theta)$.

eg. $\frac{d}{d\theta}(\theta^2 - \theta + 1) = 2\theta - 1$. $f': \mathbb{R} \rightarrow \mathbb{R}$ is a scalar-valued function.

- Multivariate function $f(\vec{\theta})$, $\vec{\theta} \in \mathbb{R}^d$, $f: \mathbb{R}^d \rightarrow \mathbb{R}$

We need to find the gradient, $\nabla_{\vec{\theta}} f(\vec{\theta})$.

$$\nabla_{\vec{\theta}} f(\vec{\theta}) = \begin{bmatrix} \frac{\partial}{\partial \theta_1} f(\vec{\theta}) \\ \frac{\partial}{\partial \theta_2} f(\vec{\theta}) \\ \vdots \\ \frac{\partial}{\partial \theta_d} f(\vec{\theta}) \end{bmatrix} \quad \nabla_{\vec{\theta}} f: \mathbb{R}^d \rightarrow \mathbb{R}^d \text{ is a vector-valued function.}$$

eg. $f(\vec{\theta}) = \begin{bmatrix} \theta_1 \\ \theta_2 \end{bmatrix} \cdot \begin{bmatrix} 1 \\ 2 \end{bmatrix} = \theta_1 + 2\theta_2$, $\nabla_{\vec{\theta}} f(\vec{\theta}) = \begin{bmatrix} 1 \\ 2 \end{bmatrix}$.

Gradient of dot product

$$\vec{\theta} \cdot \vec{x} = \theta_1 x_1 + \dots + \theta_d x_d$$

$$\frac{\partial}{\partial \theta_j} (\vec{\theta} \cdot \vec{x}) = x_j$$

$$\nabla_{\vec{\theta}} (\vec{\theta} \cdot \vec{x}) = \left[\frac{\partial}{\partial \theta_1} (\vec{\theta} \cdot \vec{x}), \dots, \frac{\partial}{\partial \theta_d} (\vec{\theta} \cdot \vec{x}) \right]^T = [x_1, x_2, \dots, x_d]^T = \vec{x}$$

$\Rightarrow$ Back to our problem at hand...

Want to minimize empirical risk, $R_N(\vec{\theta})$, with respect to $\vec{\theta}$
Using gradient descent: need to calculate the gradient.

$$\nabla_{\vec{\theta}} R_N(\vec{\theta}) = \left[\frac{\partial R_N(\vec{\theta})}{\partial \theta_1}, \frac{\partial R_N(\vec{\theta})}{\partial \theta_2}, \dots, \frac{\partial R_N(\vec{\theta})}{\partial \theta_d} \right]^T$$

But recall: $R_N(\vec{\theta}) = \frac{1}{N} \sum_{i=1}^N \text{loss}(\vec{x}^{(i)}, y^{(i)}; \vec{\theta})$

the summation means we have to look at every data point before making a single update. $\rightarrow$ Slow.

Instead, we can update $\vec{\theta}$ based on a single point.

⇒ Stochastic Gradient Descent.

$$\vec{\theta}^{(0)} = \vec{0}, k = 0$$

While convergence criteria not met

randomly shuffle data points

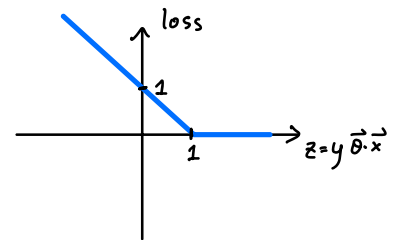
for $i = 1 \dots N$

$$\vec{\theta}^{(k+1)} = \vec{\theta}^{(k)} - \eta_k \nabla_{\vec{\theta}} \text{loss}(\vec{x}^{(i)}, y^{(i)}; \vec{\theta}) \Big|_{\vec{\theta} = \vec{\theta}^{(k)}}$$

$k++$

Q: Derive the update rule for SGD applied to hinge loss.

$$\text{loss}_h(\vec{x}^{(i)}, y^{(i)}; \vec{\theta}) = \max\{0, 1 - y^{(i)} \vec{\theta} \cdot \vec{x}^{(i)}\}$$



- if $y^{(i)} \vec{\theta} \cdot \vec{x}^{(i)} > 1$, loss is zero which is the minimum, gradient is also 0. no update.

- if $y^{(i)} \vec{\theta} \cdot \vec{x}^{(i)} \leq 1$:

$$\begin{aligned} & \nabla_{\vec{\theta}} \text{loss}_h(\vec{x}^{(i)}, y^{(i)}; \vec{\theta}) \Big|_{\vec{\theta} = \vec{\theta}^{(k)}} \\ &= \nabla_{\vec{\theta}} (1 - y^{(i)} \vec{\theta} \cdot \vec{x}^{(i)}) \Big|_{\vec{\theta} = \vec{\theta}^{(k)}} \\ &= 0 - y^{(i)} \nabla_{\vec{\theta}} (\vec{\theta} \cdot \vec{x}^{(i)}) \Big|_{\vec{\theta} = \vec{\theta}^{(k)}} \\ &= 0 - y^{(i)} \vec{x}^{(i)} \Big|_{\vec{\theta} = \vec{\theta}^{(k)}} \\ &= -y^{(i)} \vec{x}^{(i)} \end{aligned}$$

$$\Rightarrow \vec{\theta}^{(k+1)} = \vec{\theta}^{(k)} + \eta y^{(i)} \vec{x}^{(i)}$$

* Compare to perceptron update rule.

Practical Notes

► Why shuffle points?

- remove the effects of any unintended / artificial ordering of data.
e.g. sorted by class (all +ve followed by all -ve)
- faster convergence

► Convergence Criteria.

- in perception we looked at if "all points are correctly classified"
i.e. check $E_N(\vec{\theta}) = 0$
- Here, $\lim_{k \rightarrow \infty} E_N(\vec{\theta}^{(k)}) \neq 0$ since pts may not be linearly separable.
- Similarly, $\lim_{k \rightarrow \infty} R_N(\vec{\theta}^{(k)})$ won't go to zero.
- Instead, look at
 - how $R_N(\vec{\theta})$ changes. e.g. by less than ϵ .
 - how $\vec{\theta}$ changes. e.g. $\|\vec{\theta}^{(k+1)} - \vec{\theta}^{(k)}\| < \epsilon$.

► How do we set η ?

* too small : slow

* too large : overshoot & never converge.

→ a hyperparameter that can (and should) be tuned.

→ may set η as a function of k .

$$\eta_k = \frac{1}{(1+k)} \quad \text{this is just an example.}$$

other possibilities exist.

Appendix

* A "good" learning rate ensures convergence.

Robbins Monro conditions: $\sum_{k=1}^{\infty} \eta_k = \infty, \quad \sum_{k=1}^{\infty} \eta_k^2 < \infty$

with appropriate learning rate η , if $R_N(\vec{\theta})$ is convex,
(S)GD will converge to the global minimum "almost surely".

* (stochastic) gradient descent is a general algorithm
that can be applied to non-convex functions as well,
in which case it converges to a local minimum.

* technically, $R_N(\vec{\theta})$ with hinge loss is not everywhere differentiable
since it is piecewise linear. What do we do?

- when differentiable, no problem
- when sub-differentiable, choose any gradient around the kink.